챕터 5. 실시간 알림 - 주문 완료를 즉시 전달하다

며칠 뒤, 베타 테스터로 새 시스템을 써 본 동료가 떨떠름한 표정으로 오픈이를 찾아왔습니다.

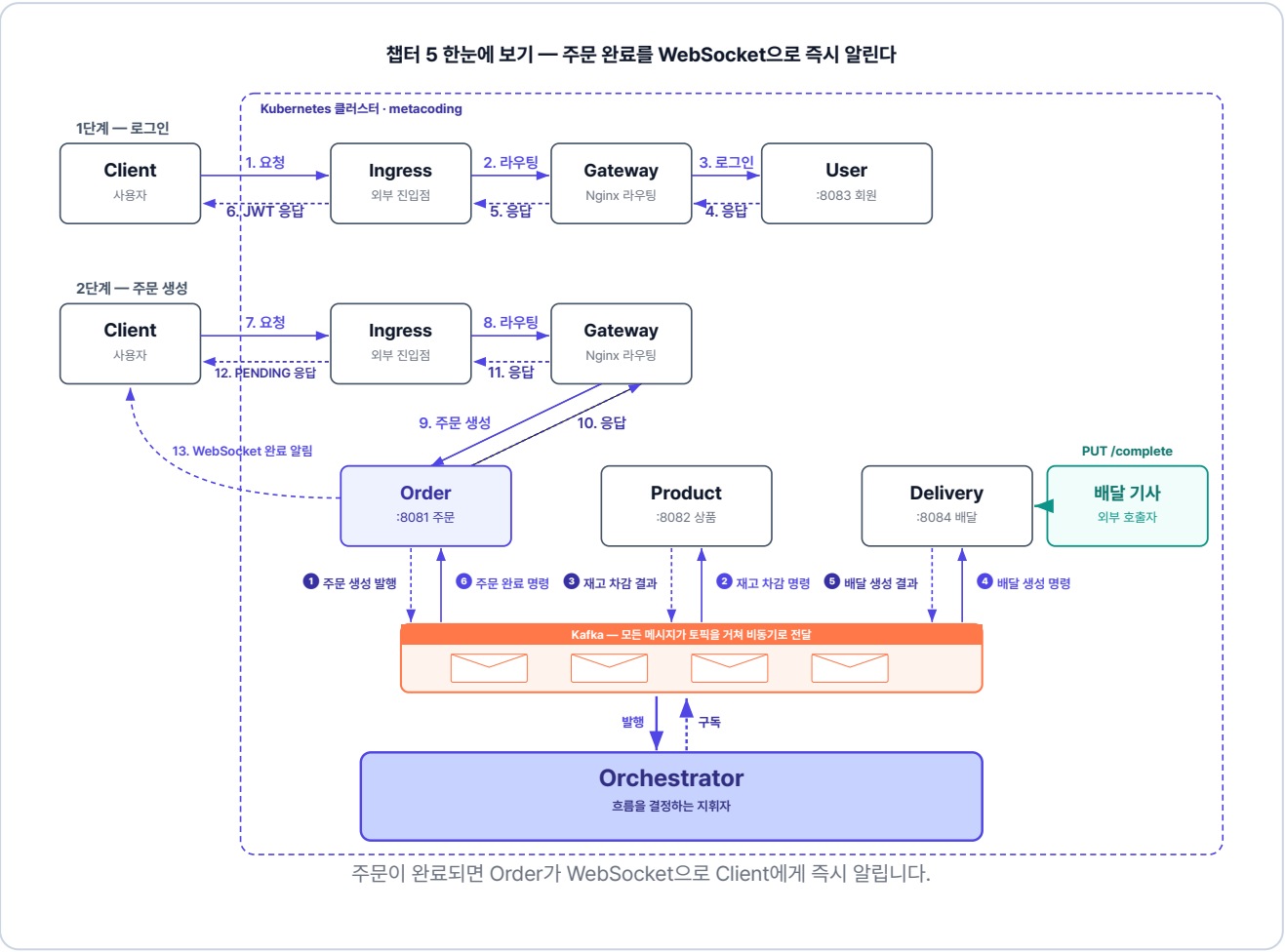
동료 : "어제 물건을 주문했는데, 화면이 계속 **처리 중**이더라고요. 끝났는지 알 수가 없어서 한참 뒤에 주문 내역을 다시 열어 보고서야 **주문 완료**된 걸 알았어요."

오픈이는 코드 흐름을 따라가 봤습니다. 주문이 생성되면 그대로 **PENDING** 상태로 응답 후, 사용자에게 **COMPLETED** 응답은 하지 않았습니다. 게다가 주문 완료는 실제 배달이 끝났는지와 무관하게, 배달이 생성되는 순간 곧바로 처리되었습니다.

이 문제를 들고 선배에게 갔습니다.

오픈이 : "서버에서 주문이 완료되었는데, 사용자는 처음 **PENDING** 상태만 응답을 받아서 주문이 완료된 사실을 알 수가 없어요. 이거는 어떻게 해결해야 하죠?"

선배 : "처리가 끝난 순간 사용자에게 알림을 줘야 해요. 서버에서 **주문 처리가 완료된 시점**을 감지해서 사용자에게 실시간으로 알려줄 방법이 필요하겠죠."



주문이 완료되면 Order가 WebSocket으로 Client에게 즉시 알립니다.

그림 5-1. 챕터 5 한눈에 보기 - 주문 완료를 웹소켓으로 즉시 알린다

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex04
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex04` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex04 디렉토리

```
ex04/
├── db/                 # MySQL
├── delivery/           # 포트 8084 (배달 완료 API 추가)
├── frontend/           # Nginx + SockJS 클라이언트 (이번 챕터 신규)
├── gateway/            # Nginx API Gateway
├── k8s/                # Kubernetes YAML 파일 (kafka·frontend 포함)
├── orchestrator/       # Kafka 워크플로우 조율
├── order/              # 포트 8081 (웹소켓 Push 추가)
├── product/            # 포트 8082
└── user/               # 포트 8083
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. 실습 순서

1. 배달 서비스에 배달 생성·완료 분리 + 배달 완료 API + `delivery-completed` 이벤트 추가
2. orchestrator에 `delivery-completed` 처리 + `delivery-created` 성공 시 대기로 변경
3. 주문 서비스에 STOMP 웹소켓 설정 + 주문 완료 시 Push
4. SockJS 기반 index.html 프론트엔드와 Nginx 프록시 구성

5. Kubernetes에 frontend 추가 배포 → 통합 시나리오 검증

5.1 웹소켓 - 폴링의 한계를 넘다

5.1.1 폴링 vs 푸시

서버에 생긴 변화를 알아내는 방법은 크게 두 가지입니다.

택배가 왔는지 확인하려고 5분마다 현관문을 열어보는 방식이 있습니다. 도착 여부는 직접 문을 열어봐야만 알 수 있습니다. 이처럼 **클라이언트가 서버에 "처리가 완료되었나요?"라고 일정 간격으로 반복해서 묻는** 방식을 **폴링(Polling)** 이라고 합니다.

반면, 택배가 도착했을 때 초인종이 울리는 방식도 있습니다. 안에 있는 사람은 문을 계속 열어볼 필요 없이, 벨이 울리는 순간 도착 사실을 알게 됩니다. 이처럼 **클라이언트가 요청하지 않아도 서버에 변화가 생겼을 때 먼저 신호를 보내는** 방식을 **푸시(Push)** 라고 합니다.

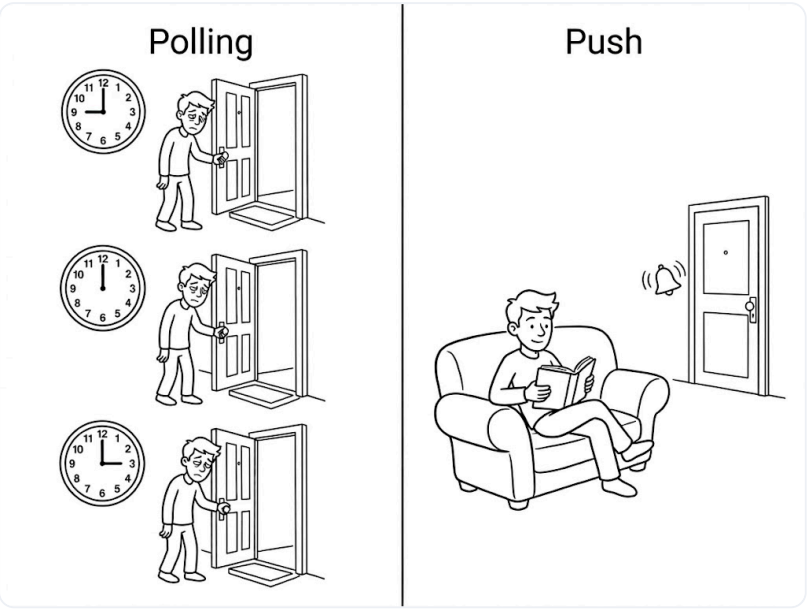


그림 5-2. 폴링 vs 푸시

폴링은 정해진 간격마다 서버에 요청을 보냅니다. 하지만 서버의 상태가 바뀌지 않았다면 의미 없는 요청과 응답을 반복하게 됩니다. 또한, 서버에 변화가 생기더라도 다음 요청 주기가 돌아올 때까지는 이를 감지할 수 없어, 설정한 간격만큼 데이터 전달이 지연되는 한계가 있습니다.

반면 푸시는 서버에 이벤트가 발생한 순간에만 신호를 보내기 때문에, 클라이언트는 지속적으로 상태를 확인하지 않고도 변경 사항을 즉시 수신할 수 있습니다.

5.1.2 웹소켓 - 실시간 양방향 통신

푸시를 구현하는 대표적인 기술이 바로 **웹소켓(WebSocket)** 입니다.

전통적인 HTTP 요청-응답 방식은 '편지'와 같습니다. 편지를 한 통 보내고 답장이 오면 한 번의 통신이 끝나며, 다음 상태가 궁금하면 다시 편지를 보내야 합니다. **클라이언트가 먼저 요청을 하지 않으면 서버는 아무것도 응답할 수 없는 구조**입니다.

반면 웹소켓은 '전화 통화'에 가깝습니다. 한 번 연결되면 끊지 않고 채널을 유지하므로, **상대가 묻지 않아도 어느 쪽이든 먼저 말을 할 수 있습니다**. 그래서 서버에 변화가 생기는 순간 알림을 클라이언트에게 보내는 것이 가능해집니다.

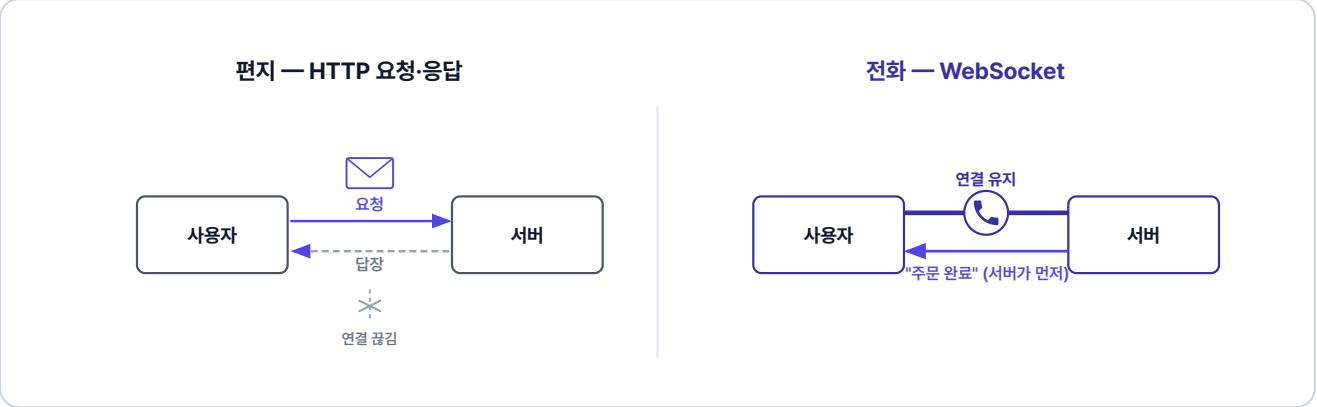


그림 5-3. 편지와 전화로 본 HTTP 요청·응답과 웹소켓

> **웹소켓(WebSocket)이란?** 클라이언트와 서버가 한 번 연결을 맺으면 이를 끊지 않고 유지하는 통신 방식입니다. 연결이 유효한 동안에는 서버가 클라이언트의 요청을 기다리지 않고도 데이터를 보낼 수 있어, 상태 변화를 실시간으로 전달할 수 있습니다.

지금은 주문이 생성됨과 동시에 완료 처리가 됩니다. 실시간 알림이 올바르게 작동하려면 실제 배달이 끝난 뒤에 주문이 완료되어야 하므로, 배달 완료 기능을 추가해야 합니다.

5.2 배달 완료 - 생성과 완료를 분리한다

이제부터 배달 생성이 발생하면 배달이 **PENDING**으로 남고, 배달 기사가 배달 완료 처리를 해야 **COMPLETED**가 됩니다. 배달이 만들어진 뒤 완료되기까지를 순서대로 보겠습니다.

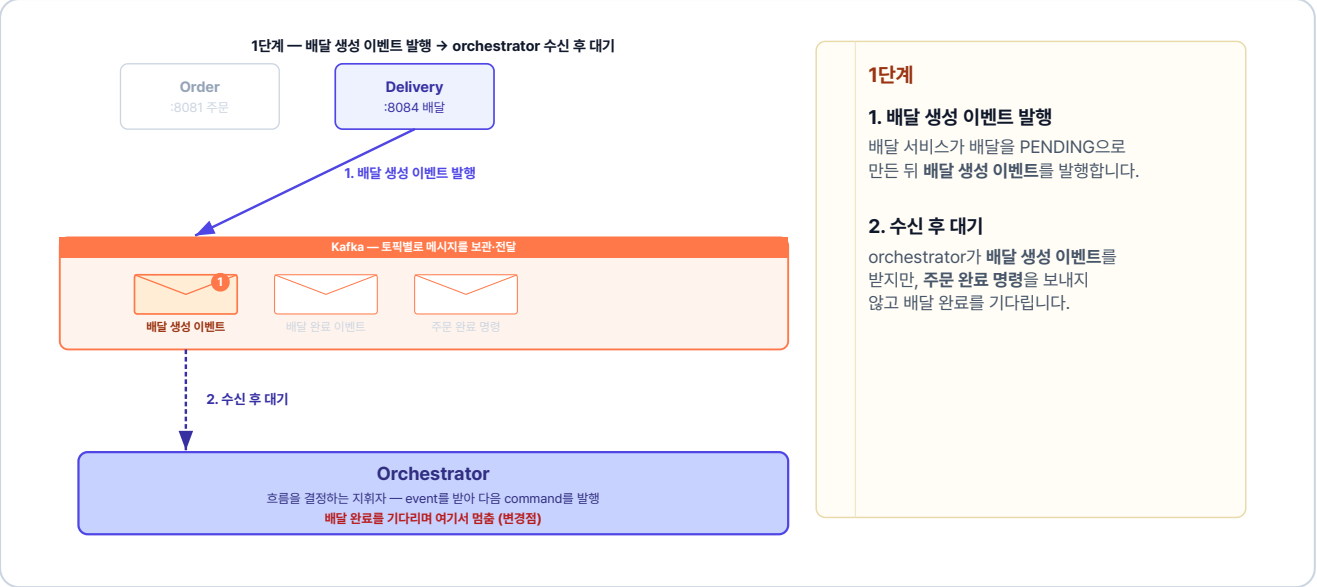


그림 5-4. 1단계 - 배달 생성 이벤트 발행 → orchestrator 수신 후 대기

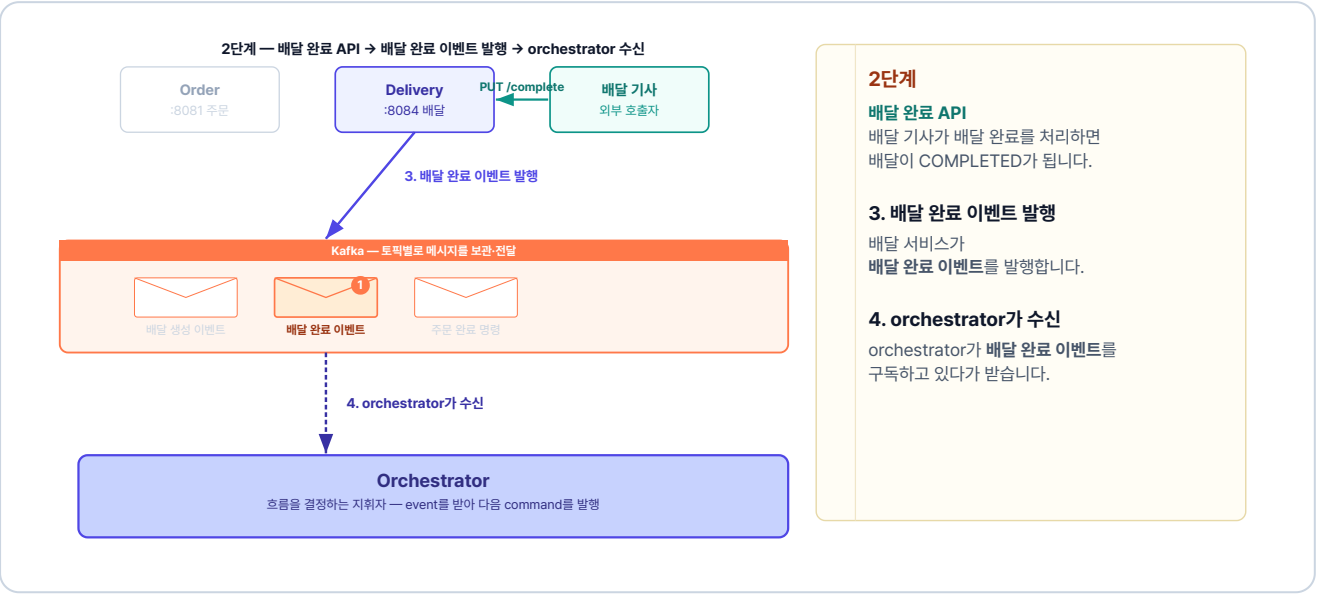


그림 5-5. 2단계 - 배달 완료 API → 배달 완료 이벤트 발행 → orchestrator 수신

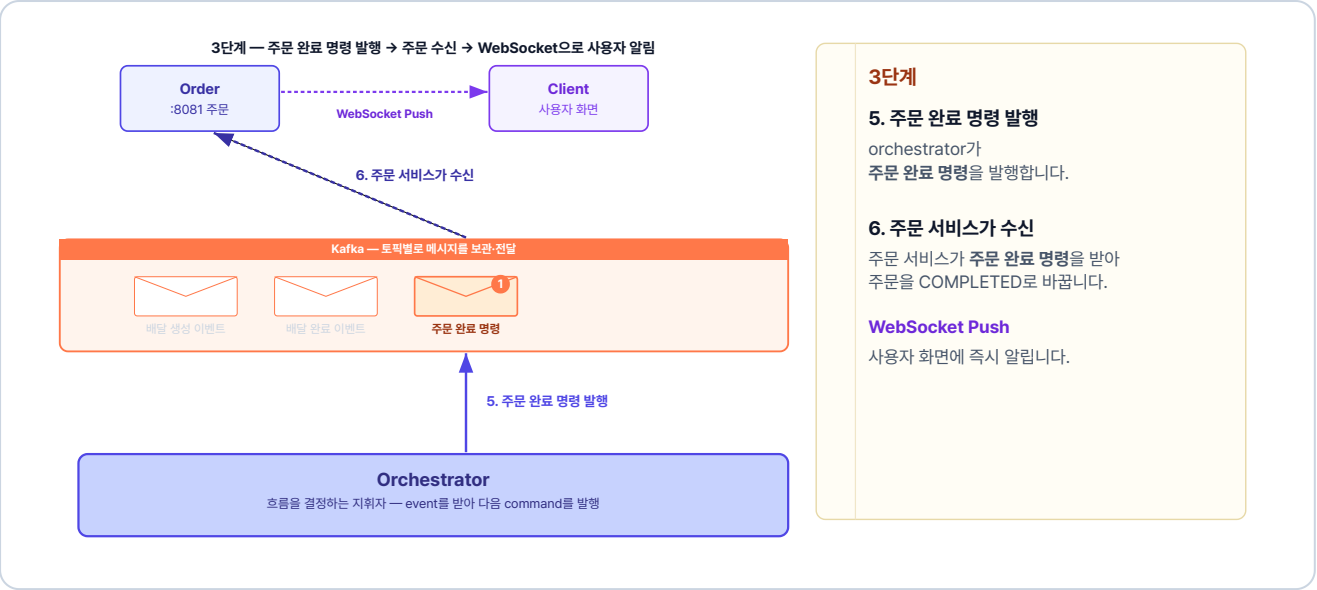


그림 5-6. 3단계 - 주문 완료 명령 발행 → 주문 수신 → 웹소켓 알림

이 세 단계가 차례로 이어지면, 배달이 실제로 완료되는 순간 사용자에게 완료 알림이 전달됩니다.

5.3 웹소켓 연결 흐름

이번에는 웹소켓 연결 흐름을 살펴보겠습니다. 주문 서비스가 보낸 알림이 사용자 화면에 뜨기까지, 크게 세 단계를 거칩니다.

5.3.1 브라우저와 주문 서비스의 연결 - HTTP에서 웹소켓으로

전화는 한쪽이 걸고 상대가 받아야 통화가 이어집니다. 마찬가지로 브라우저가 연결을 요청하고 주문 서비스가 받아들이면 **양방향 연결**이 열립니다. 이 연결은 **업그레이드 헤더**를 통해 일반 HTTP에서 **웹소켓**으로 바뀝니다.

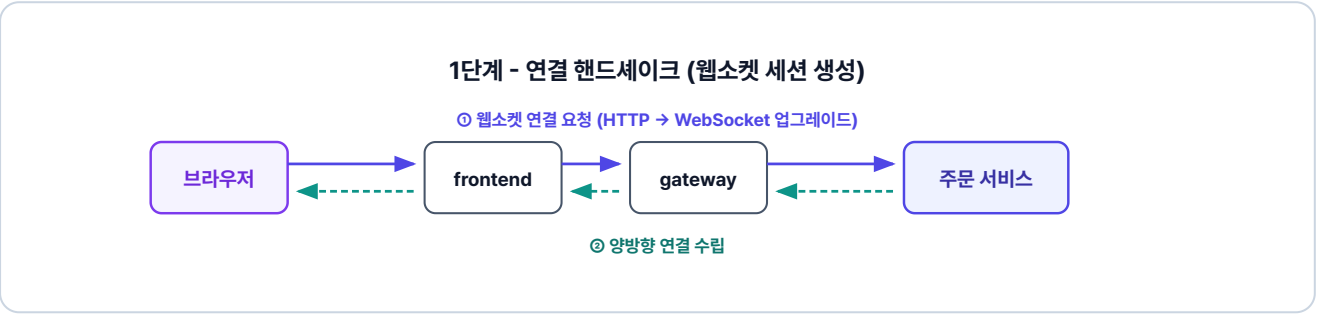


그림 5-7. 1단계 - 브라우저가 청하고 서버가 받아들여 양방향 연결이 열립니다

> **업그레이드 헤더란?** HTTP Upgrade 헤더는 클라이언트와 서버가 현재 사용 중인 HTTP 연결을 다른 프로토콜로 전환하기 위해 사용하는 헤더입니다. 주로 웹소켓 연결을 설정할 때 사용되며, 이를 통해 동일한 연결에서 새로운 통신 방식을 사용할 수 있습니다.

5.3.2 브라우저의 채널 구독 - 명부에 등록

웹소켓이 연결되더라도 서버는 누구에게 어떤 알림을 보낼지 스스로 알 수 없습니다. 따라서 브라우저가 먼저 서버에 자신이 받을 채널을 알려주며 **구독(Subscribe)** 해야 합니다.

브라우저가 특정 채널을 구독하면, 주문 서비스 안의 웹소켓 브로커는 **구독 명부**에 그 사실을 기록해 둡니다. 즉, 브라우저가 구독하고 서버가 명부를 작성해 관리하는 구조입니다. 이제 서버는 이 명부를 보고 정확한 대상에게 알림을 발송합니다.



그림 5-8. 2단계 - 브라우저가 구독하면 웹소켓 브로커가 구독 명부에 등록합니다

> **웹소켓 브로커란?** 메시지를 보내는 쪽과 받는 쪽 사이에서 전달을 중개하는 역할입니다. 어떤 클라이언트가 어떤 채널을 구독했는지 명부로 관리하다가, 메시지가 들어오면 같은 채널을 구독한 클라이언트에게 전달합니다.

5.3.3 주문 서비스의 알림 발송 - 같은 채널 찾아 전달

채널 주소에는 **알림을 받을 사용자**에 대한 정보가 들어 있습니다. 그래서 주문이 완료되면, 주문 서비스는 완료된 주문이 누구의 것인지부터 확인합니다. 그리고 주문한 사용자의 채널 주소를 구독 명부에서 찾아 알림을 보냅니다.

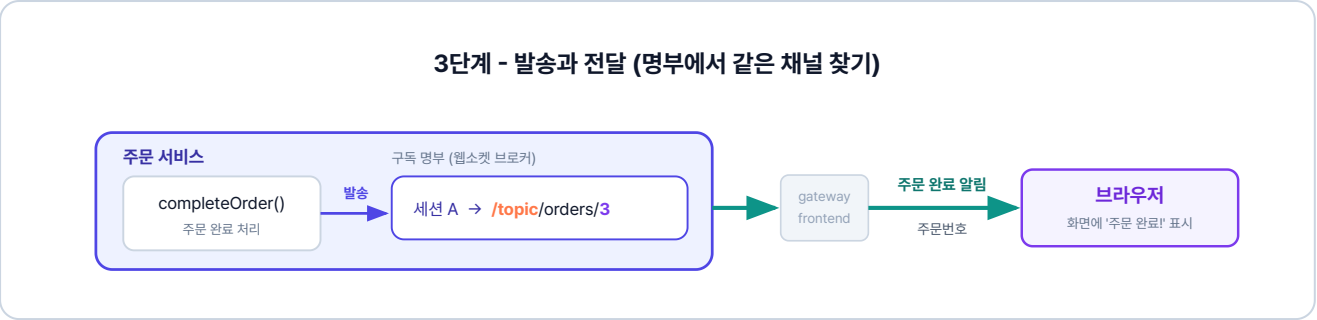


그림 5-9. 3단계 - 발송 주소와 같은 채널을 명부에서 찾아 구독한 브라우저에 보냅니다

세 단계가 모두 갖춰지면, 배달이 끝나는 순간 사용자 화면에 주문 완료가 표시됩니다.

이제 코드로 구현해 보겠습니다. 먼저 배달 서비스에서 배달의 생성과 완료 과정을 분리하는 작업부터 시작합니다.

5.4 배달 서비스 - 배달 완료 API

배달 서비스는 배달의 생성과 완료를 분리하고, 배달 기사가 호출할 배달 완료 API를 추가합니다.

5.4.1 createDelivery 수정 - 배달 생성·완료 분리

배달 생성 시 배달 완료 호출을 지우면 배달은 **PENDING**으로 남습니다. 배달 완료는 배달 기사가 직접 호출할 때까지 미뤄집니다.

`usecase/DeliveryService.java`의 `createDelivery`를 아래처럼 고칩니다.

실습 1 **배달 서비스 - usecase/DeliveryService.java. 생성 시 완료 호출 제거**

```
@Transactional
public DeliveryResponse createDelivery(int orderId, String address) {
    Delivery createdDelivery = deliveryRepository.save(
            Delivery.create(orderId, address));
    Delivery.validateAddress(address);
    // 삭제: createdDelivery.complete();  ← 생성 시 완료 호출 제거
    return DeliveryResponse.from(createdDelivery);
}
```

5.4.2 completeDelivery 추가 - 배달 완료 API

이번에는 배달 기사가 호출할 배달 완료 메서드를 추가합니다.

실습 2 **배달 서비스 - usecase/DeliveryService.java. completeDelivery 추가**

```
@Override
@Transactional
public DeliveryResponse completeDelivery(int deliveryId) {
    Delivery findDelivery = deliveryRepository.findById(deliveryId)
            .orElseThrow(() -> new Exception404("배달 정보를 조회할 수 없습니다."));
    findDelivery.complete();
    deliveryEventProducer.publishDeliveryCompleted(
            new DeliveryCompletedEvent(findDelivery.getOrderId()));
    return DeliveryResponse.from(findDelivery);
}
```

배달 기사가 배달 완료를 호출하면 배달이 완료되고, 배달 완료 이벤트가 발행됩니다. 이제 이 이벤트를 orchestrator가 받도록 수정합니다.

5.5 orchestrator - 배달 완료 이벤트 처리 추가

챕터 4에서는 배달 생성이 성공하면 orchestrator가 곧바로 주문 완료 명령을 발행했습니다. 이번에는 배달이 완료될 때 주문 완료 명령을 발행하도록 바꿉니다.

`handler/OrderOrchestrator.java`에 `deliveryCompleted` 리스너를 추가합니다.

실습 3 **오케스트레이터 서비스 - handler/OrderOrchestrator.java. 주문 완료 명령 발행**

```
@KafkaListener(topics = "delivery-completed", groupId = "orchestrator")
public void deliveryCompleted(DeliveryCompletedEvent event) {
    // 배달기사가 완료 API를 호출한 시점 → 주문 완료 명령 발행
    kafkaTemplate.send(
            "complete-order-command",
            String.valueOf(event.orderId()),
            new CompleteOrderCommand(event.orderId())
    );
}
```

5.6 주문 서비스 - STOMP로 실시간 Push 구현

마지막으로 주문 서비스가 주문 완료 명령을 받으면, 클라이언트에게 알리기 위해 웹소켓을 추가합니다.

5.6.1 웹소켓 설정

WebSocketConfig는 두 가지 주소를 등록합니다. 하나는 **클라이언트가 웹소켓으로 연결할 주소 (`/api/ws/orders`)** 이고, 다른 하나는 **서버와 클라이언트가 알림을 주고받을 주소의 접두사 (`/topic`)** 입니다.

주문 서비스의 `core/config/WebSocketConfig.java`를 열고 아래 클래스를 작성합니다.

실습 4 **주문 서비스 - core/config/WebSocketConfig.java. STOMP 웹소켓 설정**

```
@Configuration
@EnableWebSocketMessageBroker // 이 애너테이션을 붙이면 STOMP 메시징 기능이 켜집니다
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // /topic으로 시작하는 주소로 메시지가 오면,
        // 서버가 같은 주소를 구독한 클라이언트에게 전달합니다
        config.enableSimpleBroker("/topic");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // 클라이언트가 웹소켓 연결을 시작할 주소입니다. 어떤 출처에서든 연결을 허용합니다
        registry.addEndpoint("/api/ws/orders")
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }
}
```

웹소켓 위에 STOMP 프로토콜을 사용합니다. 웹소켓은 서버와 클라이언트를 계속 연결해 주지만, 연결만으로는 메시지를 누구에게 보낼지 가려내지 못합니다. 그래서 이 연결 위에 **STOMP(Simple Text Oriented Messaging Protocol)** 라는 메시징 규칙을 얹습니다. STOMP는 메시지마다 채널(주소)을 붙이고, **그 채널을 구독한 클라이언트에게만 전달하는 발행-구독 구조**를 제공합니다. 이 예제에서 서버는 `/topic/orders/{userId}` 채널로 보내고, 같은 채널을 구독한 사용자만 자기 주문 완료 알림을 받습니다.

클라이언트는 연결 주소로 웹소켓을 연 뒤, `/topic` 주소를 구독해 알림을 받습니다. 이때 웹소켓 브로커는 구독한 주소를 명부에 등록해 둡니다.

5.6.2 주문 완료 시 알림 발송

완료 알림은 `/topic/orders/{userId}` 채널로 보냅니다. 주문 완료 후 채널을 구독한 클라이언트에게 메시지를 전달합니다.

`usecase/OrderService.java`의 `completeOrder` 메서드를 아래처럼 수정합니다.

실습 5 **주문 서비스 - usecase/OrderService.java. completeOrder + WebSocket Push**

```
@Transactional
public void completeOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
            .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    findOrder.complete();
    // 추가: 이 채널을 구독한 클라이언트에게 메시지를 보냄
    messagingTemplate.convertAndSend("/topic/orders/" + findOrder.getUserId(), orderI
d);
}
```

핵심 코드 외에 주문 서비스에 필요한 설정은 깃헙 레포에서 확인합니다.

5.7 프론트엔드 연결

서버는 주문이 완료되면 채널로 알림을 보냅니다. 이제 **같은 채널을 구독해 알림을 받는 클라이언트**를 만들 차례입니다. 먼저 프록시가 웹소켓 연결을 끊지 않게 하고, 브라우저가 STOMP로 자기 채널을 구독하게 합니다.

5.7.1 업그레이드 헤더 전달

앞에서 본 업그레이드 헤더는 브라우저와 주문 서비스 사이의 frontend와 gateway를 거쳐야 합니다. 그런데 이 둘이 업그레이드 헤더를 넘기지 않으면 일반 요청처럼 처리돼 **연결이 끊깁니다**. 그래서 frontend와 gateway 두 곳의 nginx에 **업그레이드 헤더를 전달**하도록 설정합니다.

`frontend/nginx.conf`의 `/api/ws/` 위치에 아래처럼 설정합니다.

참고 **frontend/nginx.conf. 웹소켓 업그레이드 헤더**

```
location /api/ws/ {
    proxy_pass http://gateway;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

`gateway/nginx.conf`의 `/api/ws/` 블록에도 같은 코드를 넣습니다. 그래야 브라우저에서 gateway까지 업그레이드 헤더가 끊기지 않고 전달됩니다.

5.7.2 클라이언트 - STOMP 구독

`frontend/index.html`은 서버 알림을 받아 화면에 주문 완료를 표시하는 STOMP 클라이언트입니다. 주문하기 버튼을 누르면, 먼저 웹소켓을 연결하고 자기 주문 완료 알림이 올 `/topic/orders/{userId}` 채널을 구독합니다. 알림 받을 준비를 마친 다음 주문 API를 호출합니다.

참고 **frontend/index.html. STOMP 연결과 구독**

```
// 1. /api/ws/orders로 웹소켓 연결
stomp = Stomp.over(new SockJS('/api/ws/orders?token=' + TOKEN));
stomp.connect({}, function () {
    // 2. 내 주문 알림 채널 구독
    stomp.subscribe('/topic/orders/' + userId, function (msg) {
        // 3. 서버가 보낸 메시지를 화면에 표시
        const orderId = JSON.parse(msg.body);
        status.textContent = '주문 완료! (주문번호: ' + orderId + ')';
    });
});
```

클라이언트와 서버 양쪽 코드에서 웹소켓 연결 주소(`/api/ws/orders`)와 구독 채널 주소(`/topic/orders/{userId}`)를 동일하게 맞춰주어야 정상적으로 알림이 전달됩니다. 전체 index.html은 깃헙 레포에서 확인합니다.



그림 5-10. 웹소켓 주소 일치 - 같은 색은 글자까지 똑같아야 동작합니다

5.8 전체 시스템 통합 테스트

이제 전체 시스템을 실행해, 주문 생성부터 완료 알림까지 전체 흐름을 확인합니다.

5.8.1 Kubernetes 리소스 정의

이전 챕터까지의 구성에서 frontend 서비스가 새로 추가됩니다. `k8s/frontend/` 폴더에 Deployment, Service, Ingress가 정의되어 있습니다.

파일	역할
frontend-deploy.yml	Nginx 기반 프론트엔드 Pod
frontend-service.yml	클러스터 내부 접근용 Service
frontend-ingress.yml	외부 요청을 frontend-service로 라우팅

이번 챕터부터 Ingress는 gateway-service 대신 **frontend-service**로 요청을 보냅니다. 프론트엔드 Nginx가 정적 파일을 직접 응답하고, `/api/` 요청만 gateway-service로 전달합니다.

5.8.2 이미지 빌드

Minikube 내부에 이미지를 빌드합니다.

터미널 **이미지 빌드**

```
minikube image build -t metacoding/db:3 ./db
minikube image build -t metacoding/gateway:3 ./gateway
minikube image build -t metacoding/order:3 ./order
minikube image build -t metacoding/product:3 ./product
minikube image build -t metacoding/user:3 ./user
minikube image build -t metacoding/delivery:3 ./delivery
minikube image build -t metacoding/orchestrator:3 ./orchestrator
minikube image build -t metacoding/frontend:3 ./frontend
```

5.8.3 배포

Kafka를 먼저 배포하고, 준비될 때까지 기다린 다음 나머지를 배포합니다.

터미널 **배포 순서 (Kafka 우선)**

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/gateway
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/orchestrator
kubectl apply -f k8s/frontend

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

모든 Pod가 Running 상태가 될 때까지 대기합니다.

터미널 **Pod 상태 확인**

```
kubectl get pods -n metacoding
```

kubectl get pods -n metacoding

```
NAME                                READY   STATUS    AGE
kafka-deploy-7d4c8b9f5-2xk9p        1/1     Running   2m
db-deploy-6f9b7c4d8-m4t2q           1/1     Running   90s
gateway-deploy-5c8d6f7b9-h7w3r      1/1     Running   88s
order-deploy-8b7f6c9d4-q2k8m        1/1     Running   85s
product-deploy-7c9d8b6f5-x4r2t      1/1     Running   83s
user-deploy-6d8c7b9f4-p3m9k         1/1     Running   80s
delivery-deploy-9f7c8b6d5-t6w2x     1/1     Running   78s
orchestrator-deploy-8c6f9b7d4-k9m4q 1/1     Running   75s
frontend-deploy-7b9c6d8f5-w3k2m     1/1     Running   72s

9개 Pod Running (frontend 추가)
```

그림 5-11. Pod 상태 확인

5.8.4 서비스 접근

외부에서 Ingress로 접속하기 위해 `minikube tunnel`을 실행합니다.

터미널 **외부 접근 터널**

```
minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80`로 프론트엔드에 접속할 수 있습니다.

5.8.5 통합 테스트 시나리오

Step 1: 웹소켓 연결 및 주문 생성

브라우저로 index.html에 접속합니다. 그다음 로그인 API(`POST /login`)에서 발급받은 JWT 토큰을 입력하여 웹소켓을 연결합니다.

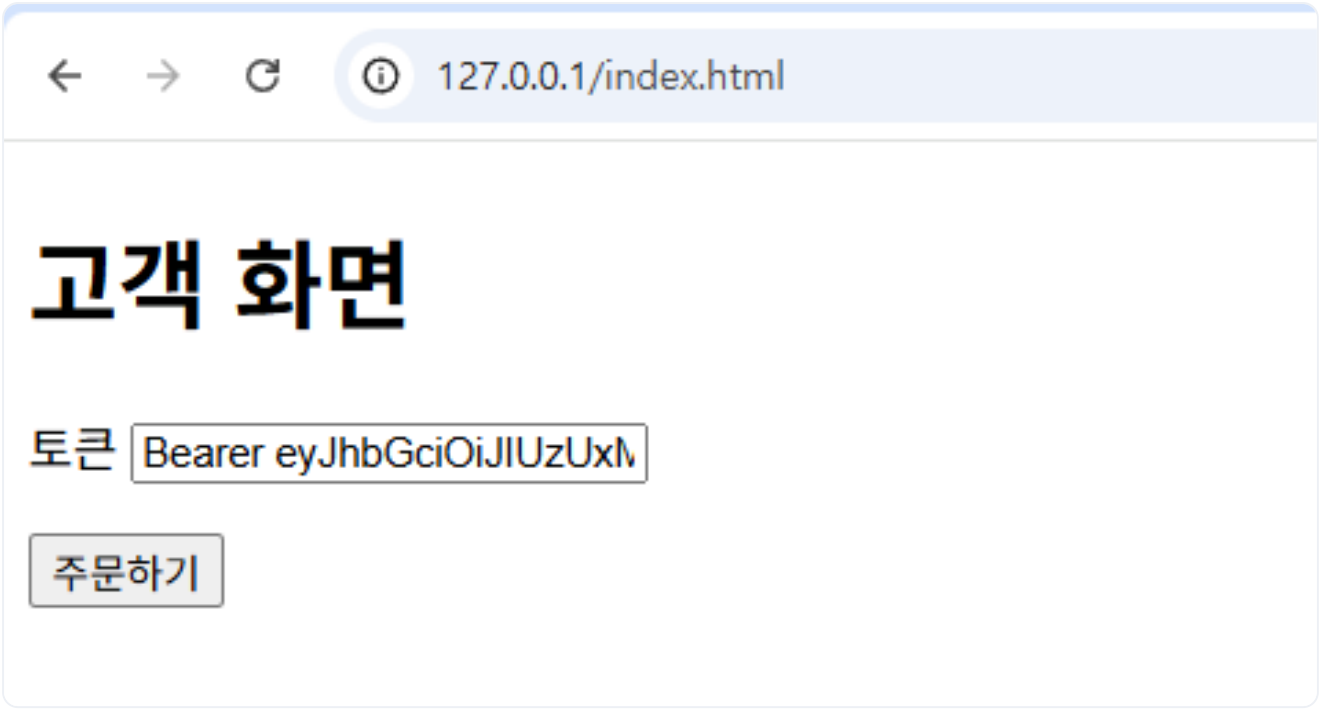


그림 5-12. 브라우저에서 index.html 접속 화면

주문하기 버튼을 클릭합니다. 그러면 index.html이 웹소켓에 연결하고 `/topic/orders/{userId}` 채널을 구독합니다.

자동 전송 **index.html이 보내는 주문 요청**

```
POST /api/orders

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

이 요청은 주문하기 버튼을 누르면 index.html이 자동으로 보내므로, 직접 호출하지 않아도 됩니다.

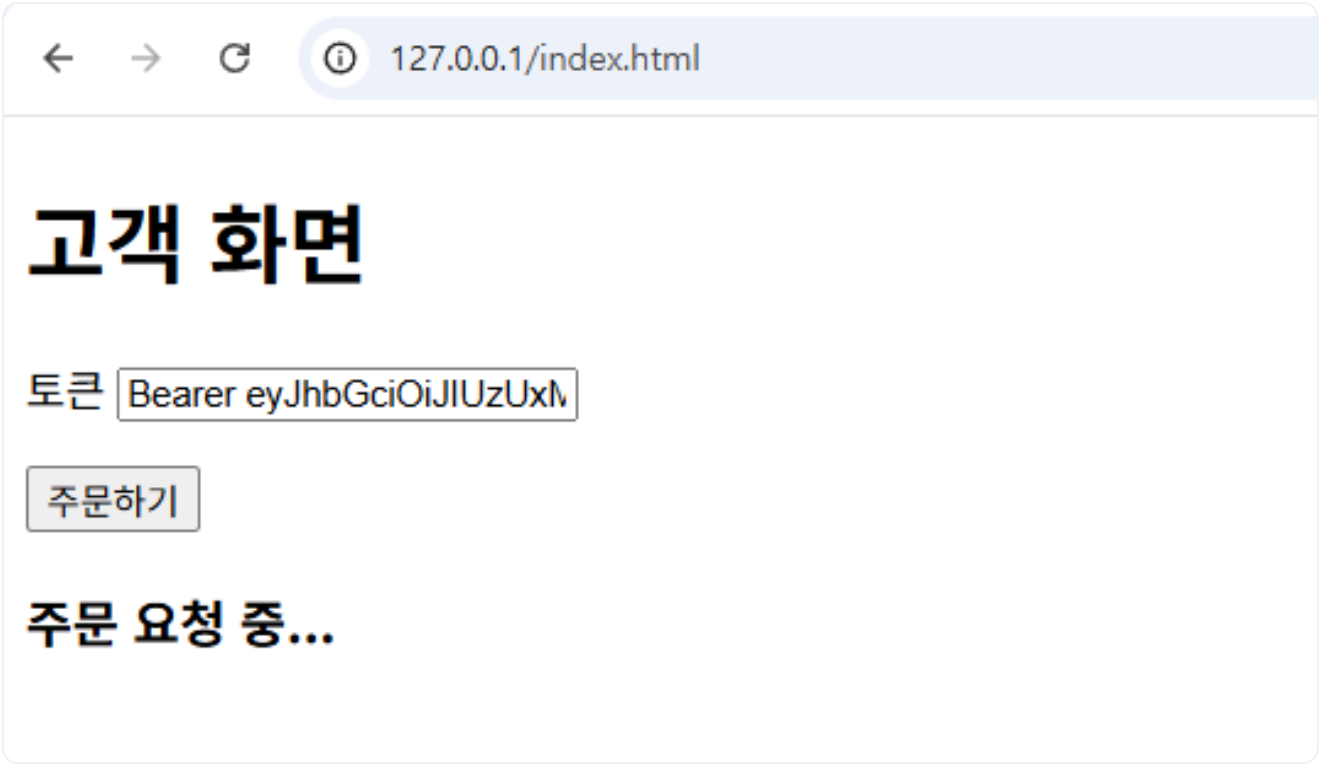


그림 5-13. 토큰 입력 후 주문하기 버튼 클릭

브라우저 `F12` - `Console`에서 웹소켓이 연결됨을 확인할 수 있습니다.

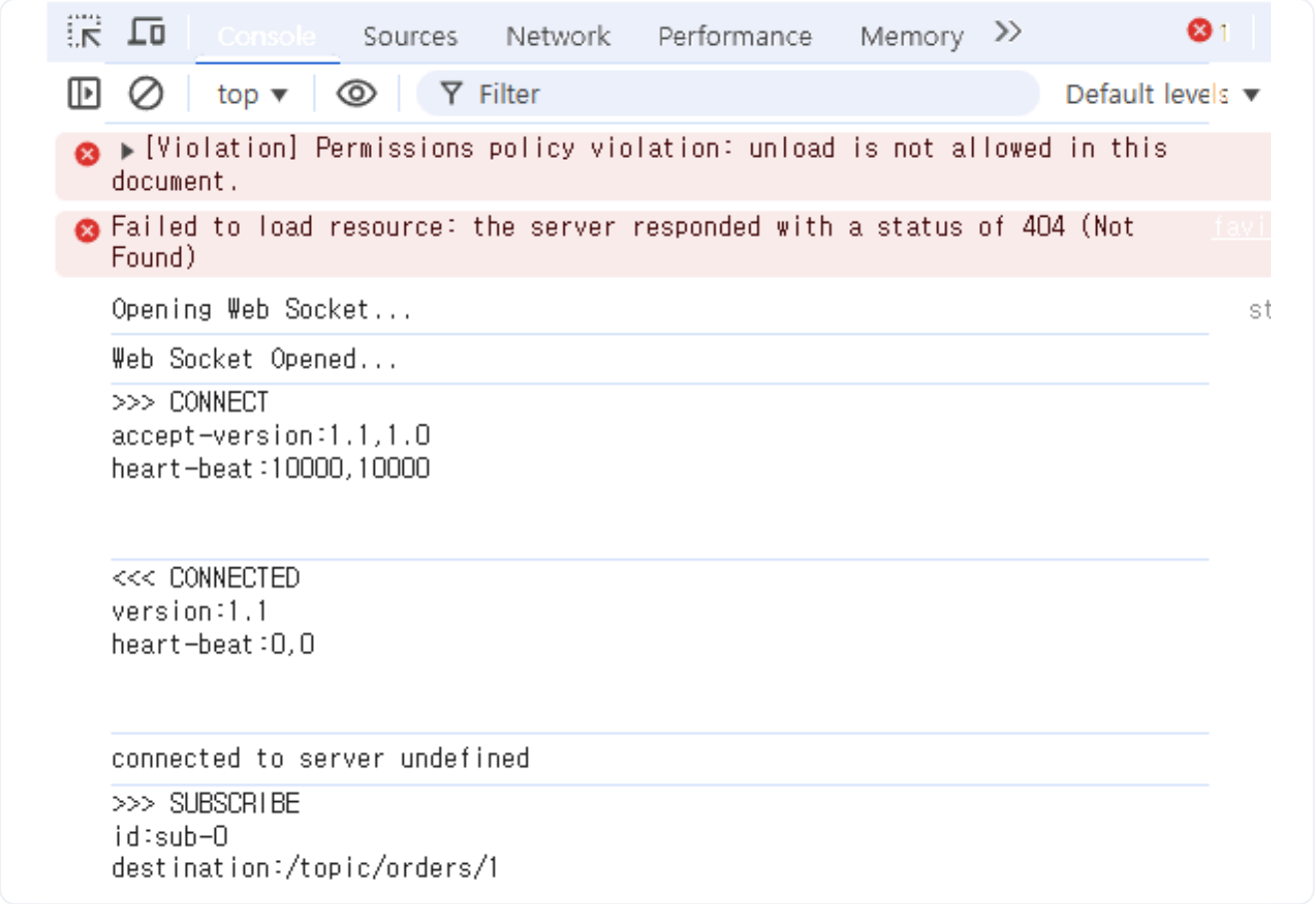


그림 5-14. 브라우저 Console에서 웹소켓 연결 확인

Hoppscotch로 생성된 주문을 확인하면 `PENDING` 상태로 머물러 있습니다.

Hoppscotch **주문 조회**

```
GET http://127.0.0.1:80/api/orders/4
```



그림 5-15. 주문 조회 결과 - PENDING 상태

Step 2: 배달 완료

먼저 생성된 배달을 확인해보겠습니다.

Hoppscotch **배달 조회**

```
GET http://127.0.0.1:80/api/deliveries/4
```



그림 5-16. 배달 조회 결과 - PENDING 상태

배달 ID가 4인 배달이 `PENDING` 상태로 생성되었습니다.

배달 완료 API를 호출해 완료 처리를 합니다.

Hoppscotch 배달 완료 호출

```
PUT http://127.0.0.1:80/api/deliveries/4/complete
```

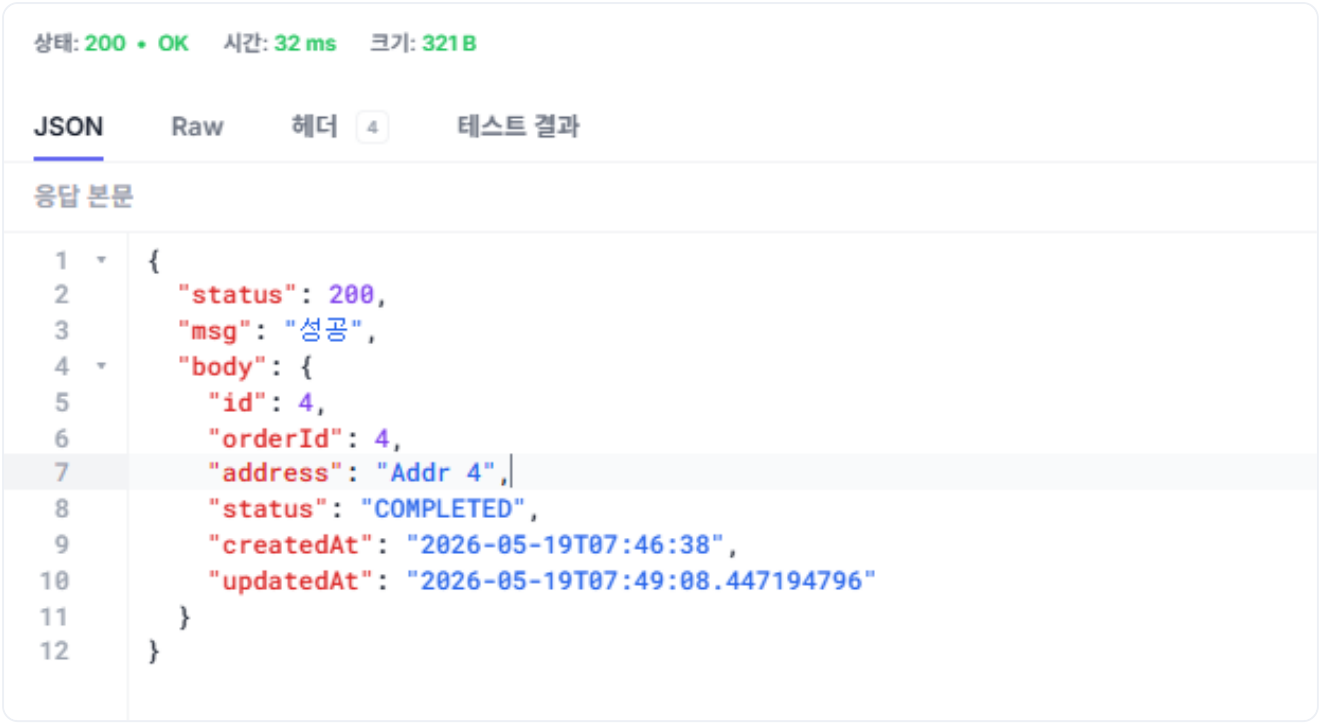


그림 5-17. 배달 완료 API 호출 결과 - COMPLETED 상태

배달 완료 버튼을 눌렀다. 이제 주문도 바뀌었을까?

Step 3: 주문 완료 및 웹소켓 응답 확인

배달 완료 처리 후, 주문 완료 명령으로 주문이 `COMPLETED` 상태가 됩니다.

Hoppscotch 주문 조회

```
GET http://127.0.0.1:80/api/orders/4
```



그림 5-18. 주문 조회 결과 - COMPLETED 상태

주문이 완료되면 웹소켓이 클라이언트에게 주문 완료 메시지를 전송합니다. 웹소켓 응답을 수신하면 클라이언트 화면이 주문 완료 상태로 변경됩니다.



그림 5-19. 웹소켓 알림 수신 - 클라이언트 화면에 주문 완료 표시

완성한 화면을 동료에게 보여 줬습니다. 주문하기를 누르자 잠시 뒤 '처리 중'이 '주문 완료'로 바뀌었습니다.

동료 : "이제 사용자도 새로고침 없이 주문이 끝난 걸 바로 알 수 있겠네요."

하나로 구성됐던 서비스를 기능별로 분리하고, 사용자에게 실시간으로 알리는 웹소켓까지 더했습니다. 이제 한쪽 서비스에 부하가 몰려도 전체가 멈추지 않고, 주문이 끝나는 순간 사용자 화면에 완료가 바로 표시됩니다.

이것만은 기억하자

- → **폴링**은 클라이언트가 변화를 계속 확인해야 하지만, **웹소켓**은 서버가 변화 순간 먼저 알립니다.
- → 배달 생성과 배달 완료를 분리해, 주문 완료를 **실제 배달이 끝난 시점**에 맞춥니다.
- → 주문이 완료되면 주문 서비스가 **웹소켓**으로 사용자에게 실시간으로 알립니다.